

# Trabajo practico

## Integrador

Modulo 3

Nelson Gonzalez

Profesores  
Fecha de entrega

Diego Wolf / Tatiana Tablada  
15/06/2026

## Respuestas

Ejercicio 1: Escribir un programa que dado el ingreso de un número retorne si el mismo es primo o no.

### Objetivos

- Desarrollar un programa que permita determinar si un número ingresado por el usuario es primo o no, aplicando estructuras de control, operadores matemáticos y validaciones lógicas.

### Entorno

- Desarrollo: Visual Studio Code
- Sistema Operativo: Win 10
- Lenguaje: Python 3

### Código

```
1  #Solicita al usuario que ingrese un numero.
2  numero = int(input("Ingrese un número: "))
3  #Una variable booleana para determinar si el numero es primo o no.
4  es_primo = True
5  #verifica si el numero es menor o igual a 1, en caso de que sea así, el numero no es primo.
6  if numero <= 1:
7      es_primo = False
8  #Recorre todos los numeros desde el 2 hasta el numero ingresado, verifica si el numero es
9  #divisible por alguno de esos numeros, si es así, el numero no es primo y se detiene el ciclo.
10 else:
11     for i in range(2, numero):
12         if numero % i == 0:
13             es_primo = False
14             break
15 #Break (Finaliza el ciclo inmediatamente al encontrar un divisor)
16 #Recorre todo los posibles divisores hasta el numero anterior a ingresar
17 if es_primo:
18     print("El número es primo.")
19 else:
20     print("El número no es primo.")
21 #Imprime el resultado indicando si el numero es primo o no.1
```

### Evidencias

-Se verifica el perfecto funcionamiento del programa mediante algunos casos de prueba:

```
(venv) PS C:\Users\ryzen\PPP> & c:\Users\ryzen\PPP\venv\Scripts\python.exe c:/Users/ryzen/PPP/Primos.py
Ingrese un número: 11
El número es primo.
(venv) PS C:\Users\ryzen\PPP> & c:\Users\ryzen\PPP\venv\Scripts\python.exe c:/Users/ryzen/PPP/Primos.py
Ingrese un número: 17
El número es primo.
(venv) PS C:\Users\ryzen\PPP> & c:\Users\ryzen\PPP\venv\Scripts\python.exe c:/Users/ryzen/PPP/Primos.py
Ingrese un número: 8
El número no es primo.
```

- Casos de prueba tabla simple

| ID CASO | DESCRIPCIÓN               | KEYS | RESULTADO ESPERADO    |
|---------|---------------------------|------|-----------------------|
| D-01    | Verificar número primo    | 11   | El número es primo    |
| D-02    | Verificar número primo    | 17   | El número es primo    |
| D-03    | Verificar número no primo | 8    | El número no es primo |

Ejercicio 2: Escribir una función que, dado el ingreso de 3 variables (a, b, c), retorne las raíces resultantes de una ecuación cuadrática.

The diagram shows the quadratic formula:  $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ . Annotations include:

- A red box around the  $\pm$  symbol with a dotted arrow pointing to it and the text: "Indica que tienes que sumar y restar, ¡entonces puede haber 2 soluciones!".
- A blue box around the entire fraction with a dotted arrow pointing to it and the text: "Variable o incógnita".
- A section titled "Discriminante" with the text: "Cuando lo resuelves el resultado nos indica el tipo de solución de la ecuación:". Below this are three bullet points:
  - ✓ Si es positivo → Hay **2** soluciones
  - ✓ Si es 0 → Hay **1** solución
  - ✓ Si es negativo → **NO** hay solución

## Objetivos

-Desarrollar un programa que permita calcular las raíces de una función cuadrática a partir de los coeficientes a, b, c.

## Entorno

- Desarrollo: Visual Studio Code
- Sistema Operativo: Win 10
- Lenguaje: Python 3

- Casos de prueba tabla simple

| ID CASO | DESCRIPCIÓN           | KEYS            | RESULTADO ESPERADO         |
|---------|-----------------------|-----------------|----------------------------|
| D-01    | Dos soluciones reales | a= 1 b= -5 c= 6 | x1 = 3<br>x2 = 2           |
| D-02    | Una sola solución     | a= 1 b= -4 c= 4 | X = -2                     |
| D-03    | Sin soluciones reales | a= 1 b= 4 c=5   | No tiene soluciones reales |

## Evidencias

- D-01

```
Ingrese el valor de a: 1
Ingrese el valor de b: -5
Ingrese el valor de c: 6
La ecuación tiene dos soluciones:
x1 = 3.0
x2 = 2.0
```

- D-02

```
Ingrese el valor de a: 1
Ingrese el valor de b: -4
Ingrese el valor de c: 4
La ecuacion tiene una unica solución:
x = 2.0
```

- D-03

```
Ingrese el valor de a: 1
Ingrese el valor de b: 4
Ingrese el valor de c: 5
La ecuacion no tiene soluciones reales
```

## Código

```
1  #importamos la libreria matematica necesaria
2  import math
3  #Defina una funcion para recibir los tres coeficientes
4  def calcular_raices(a, b, c):
5      #Calcula el discriminante usando formula matematica
6      discriminante = b**2 - 4*a*c
7      if discriminante > 0:
8          #Verifica si existen dos soluciones reales
9          x1 = (-b + math.sqrt(discriminante)) / (2*a)
10         x2 = (-b - math.sqrt(discriminante)) / (2*a)
11         #math.sqrt(discriminante) - Obtiene la raiz cuadrada del discriminante
12         print("La ecuación tiene dos soluciones:")
13         print("x1 =", x1)
14         print("x2 =", x2)
15     #Verifica si existe una unica solución
16     elif discriminante == 0:
17         x = -b / (2*a)
18         print("La ecuacion tiene una unica solución:")
19         print("x =", x)
20     #Si el discriminante es negativo o no existen raices reales se ejecuta
21     else:
22         print("La ecuacion no tiene soluciones reales")
23     #Input para pedir al usuario que ingrese lo valores de los discriminantes
24     a = float(input("Ingrese el valor de a: "))
25     b = float(input("Ingrese el valor de b: "))
26     c = float(input("Ingrese el valor de c: "))
27     #Calculo matematico de raices
28     calcular_raices(a, b, c)
```

Ejercicio 3: Automatizar los siguientes casos de prueba. Luego de que sean automatizados, deben ser subidos a un repositorio git, se debe generar el archivo y debe retornar un reporte HTML con los resultados de la ejecución.

1) Sitio: <https://www.saucedemo.com/>

## Caso 1

- El usuario se loguea al sitio como usuario standard user
- Ordenar los elementos por “price (low to high)”
- Verificar que los elementos estén ordenados

## Objetivos

- Validar que el listado de productos pueda ordenarse correctamente por precio de menor a mayor.

## Entorno

- Sitio Web: <https://www.saucedemo.com/>
- Desarrollo: Visual Studio Code
- Sistema Operativo: Windows 10
- Lenguaje: Python 3
- Framework de Automatización: Selenium WebDriver
- Framework de Testing: Pytest
- Reportes: Pytest HTML Report

## Evidencias

|                    |                                                                                                                     |
|--------------------|---------------------------------------------------------------------------------------------------------------------|
| Resultado Esperado | Ingresa correctamente el usuario y Los productos deben visualizarse ordenados desde el precio más bajo al más alto. |
| Resultado Obtenido | Se ingresa correctamente y los productos son ordenamos por precio                                                   |

```
(venv) PS C:\Trabajos-python\TPI-modulo 3- Nelson Gonzalez-comisión 2->pytest Caso1_EJ3.py -v -s
collected 1 item

Caso1_EJ3.py::test_caso1 Login realizado correctamente
Los precios se ordenaron de menor a mayor
Precios: [7.99, 9.99, 15.99, 15.99, 29.99, 49.99]
los productos están ordenados correctamente
PASSED

===== 1 passed in 6.71s =====
```

## Caso 2

- El usuario se loguea al sitio como usuario standard user
- Incorporar al carrito todos los elementos

- Ir al carrito
- Verificar que todos los elementos estén en el carrito
- Ir al checkout
- Ingresar nombre y clicar “Continue”
- Verificar que aparece el error “Error: Last Name is required”
- Ingresar un apellido y clicar “Continue”
- Verificar que aparece el error “Error: Postal Code is required”

## Objetivos

- Validar los mensajes de error durante el proceso de checkout cuando faltan datos obligatorios.

## Entorno

- Sitio Web: <https://www.saucedemo.com/>
- Desarrollo: Visual Studio Code
- Sistema Operativo: Windows 10
- Lenguaje: Python 3
- Framework de Automatización: Selenium WebDriver
- Framework de Testing: Pytest
- Reportes: Pytest HTML Report

## Evidencias

|                    |                                                              |
|--------------------|--------------------------------------------------------------|
| Resultado Esperado | Visualización de los mensajes de error correspondientes.     |
| Resultado Obtenido | Se visualizan los mensajes de error asignados por el sistema |

```
(venv) PS C:\Trabajos-python\TPI-modulo 3- Nelson Gonzalez-comisión 2-> pytest Caso2_EJ3.py -v -s
plugins: html-4.2.0, metadata-3.1.1
collected 1 item

Caso2_EJ3.py::test_caso2 Se agregaron 6 productos al carrito
Error apellido necesario
Error codigo postal necesario
PASSED

===== 1 passed in 12.21s =====
```

## Caso 3

- El usuario se loguea al sitio como usuario standard user
- Agregar un elemento al carrito

- Ir al carrito
- Remover el artículo
- Verificar que el sitio no tiene artículos agregados
- Ir a “Continue Shopping”
- Agregar 2 elementos
- Ir al carrito
- Verificar que los elementos existan
- Hacer el checkout
- Finalizar la compra
- Verificar que la compra fue realizada

## Objetivos

- Validar el flujo completo de compra incorporando productos al carrito, eliminando artículos y finalizando una compra.

## Entorno

- Sitio Web: <https://www.saucedemo.com/>
- Desarrollo: Visual Studio Code
- Sistema Operativo: Windows 10
- Lenguaje: Python 3
- Framework de Automatización: Selenium WebDriver
- Framework de Testing: Pytest
- Reportes: Pytest HTML Report

## Evidencias

|                    |                                                                                                                                                                                                       |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Resultado Esperado | <ul style="list-style-type: none"> <li>• Login correcto</li> <li>• Validación de carrito vacío</li> <li>• Validación de dos productos en el carrito</li> <li>• Mensaje de “Compra Exitosa”</li> </ul> |
| Resultado Obtenido | La compra fue procesada correctamente, el ingreso exitoso, el carrito se mostró vacío y con dos productos                                                                                             |

```
(venv) PS C:\Trabajos-python\TPI-modulo 3- Nelson Gonzalez-comisión 2-> pytest Caso3_EJ3.py -v -s
collected 1 item

Caso3_EJ3.py::test_caso3 Login exitoso
Carrito vacío
Carrito con dos productos
Compra exitosa
PASSED

===== 1 passed in 6.10s =====
```

2) - Sitio: Poke Api (<https://pokeapi.co/api/v2>)

## Caso 1

Nelson Gonzalez

- Hacer un get a berry/1
- Verificar que el size sea 20
- Verificar que el soil\_dryness sea 15
- Verificar que en firmness, el name sea soft.

## Objetivos

-Automatizar pruebas sobre distintos endpoints de la API pública PokeAPI., verificar la integridad y consistencia de los datos devueltos por la API, validar atributos específicos mediante aserciones automáticas y generar evidencia de ejecución mediante Pytest.

## Entorno

- API: PokeAPI
- [PokeAPI Oficial](#)
- Desarrollo: Visual Studio Code
- Sistema Operativo: Windows 10
- Lenguaje: Python 3
- Librería HTTP: Requests
- Framework de Testing: Pytest

## Evidencias y EndPoint

- <https://pokeapi.co/api/v2/berry/1>

- Casos de prueba tabla simple

- Los valores devueltos por la API deben coincidir exactamente con los definidos en la consigna.

API-01: =20

API-02: =15

API-03: = soft

| ID     | Datos         | Resultado |
|--------|---------------|-----------|
| API-01 | size          | 20        |
| API-02 | Soil_dryness  | 15        |
| API-03 | firmness.name | soft      |

```
(venv) PS C:\Trabajos-python\TPI-modulo 3- Nelson Gonzalez-comisión 2-> pytest Sitio2_EJ1.py -v -s
collected 1 item

Sitio2_EJ1.py::test_caso1_api Respuesta exitosa de la API
Validación correcta: size = 20
Validación correcta: soil_dryness = 15
Validación correcta: firmness.name = soft
PASSED

===== 1 passed in 1.31s =====
```

## Caso 2

- Hacer un get a berry/2
- Verificar que en firmness, el name sea super-hard



- Verificar que el size sea mayor al del punto anterior
- Verificar que el soil\_dryness sea igual al del punto anterior

## Objetivos

-Automatizar pruebas sobre distintos endpoints de la API pública PokeAPI, verificar la integridad y consistencia de los datos devueltos por la API, validar atributos específicos mediante aserciones automáticas y generar evidencia de ejecución mediante Pytest.

## Entorno

- **API:** PokeAPI
- **PokeAPI Oficial**
- **Desarrollo:** Visual Studio Code
- **Sistema Operativo:** Windows 10
- **Lenguaje:** Python 3
- **Librería HTTP:** Requests
- **Framework de Testing:** Pytest

## Evidencias y EndPoint

-<https://pokeapi.co/api/v2/berry/2>

- Casos de prueba tabla simple

-Los valores devueltos por la API tienen que cumplir con las condiciones de comparación definidas

API-04: berry2 = "super hard"

API-05: [size]berry2 > [size]berry1

API-06: soil\_dryness = 15

| ID     | Datos         | Resultado    |
|--------|---------------|--------------|
| API-04 | Firmness.name | super hard   |
| API-05 | size          | > que berry1 |
| API-06 | Soil_dryness  | = a berry1   |

```
(venv) PS C:\Trabajos-python\TPI-modulo 3- Nelson Gonzalez-comisión 2-> pytest Sitio2_EJ2.py -v -s
collected 1 item

Sitio2_EJ2.py::test_caso2_api Consultas realizadas correctamente
Correcto, firmness = super-hard
Correcto, size = 80 es mayor que 20
Correcto, soil_dryness = 15
PASSED

===== 1 passed in 1.85s =====
```

## Caso 3

- Hacer un get a pikachu (<https://pokeapi.co/api/v2/pokemon/pikachu/>)
- Verificar que su experiencia base es mayor a 10 y menor a 1000

- Verificar que su tipo es “electric”

## Objetivos

-Automatizar pruebas sobre distintos endpoints de la API pública PokeAPI, verificar la integridad y consistencia de los datos devueltos por la API, validar atributos específicos mediante aserciones automáticas y generar evidencia de ejecución mediante Pytest

## Entorno

- **API:** PokeAPI
- **PokeAPI Oficial**
- **Desarrollo:** Visual Studio Code
- **Sistema Operativo:** Windows 10
- **Lenguaje:** Python 3
- **Librería HTTP:** Requests
- **Framework de Testing:** Pytest

## Evidencias y EndPoint

-<https://pokeapi.co/api/v2/berry/2>

- Casos de prueba tabla simple

- La experiencia base debe encontrarse dentro del rango definido y el tipo principal debe coincidir con el valor esperado.

```
(venv) PS C:\Trabajos-python\TPI-modulo 3- Nelson Gonzalez-comisión 2-> pytest Sitio2_EJ3.py -v -s
plugins: html-4.2.0, metadata-3.1.1
collected 1 item

Sitio2_EJ3.py::test_caso3_api Si es pikachu :D
Experiencia base validada: 112
Tipo: electric
PASSED

===== 1 passed in 0.97s =====
```

| ID     | Datos           | Resultado |
|--------|-----------------|-----------|
| API-07 | base_experience | > a 10    |
| API-08 | base_experience | < a 1000  |
| API-09 | Type.name       | electric  |